VCL Project Report

VCL Project Report
- 1. 选题：SVG Renderer (⭐ ⭐)
- 2. 实现思路
  - 1）SVG 文件格式
  - 2）CaseSVG.h 和 CaseSVG.cpp
  - 3）主逻辑：SVG 渲染
    - SVG DOM
    - 解析属性
    - 几何构造
    - 曲线采样
    - 光栅化
    - 像素写入
- 3. 实现结果
- 4. 运行

1. 选题：SVG Renderer (⭐ ⭐)

2. 实现思路

1）SVG 文件格式

这是基于 XML 的二维矢量图形格式，按照“标签”组织；本质上也是一个 XML 文档，因此可以通过常见的 XML 解析器（本次project中利用 `tinyxml2` 辅助解析）进行读取和遍历，形如：

```
<svg width="200" height="200" viewBox="0 0 200 200">
    <rect x="10" y="10" width="50" height="50" fill="red"/>
    <circle cx="100" cy="100" r="40" stroke="blue" fill="none"/>
    <path d="M 10 150 L 190 150" stroke="black"/>
</svg>
```

SVG 文件的基本结构有：根元素 `<svg>`，基本图形元素（定义各种形状如矩形 `<rect>`、圆形 `<circle>`、线条 `<line>`、路径 `<path>` 等）， 样式与属性 如 `fill`（填充颜色）、`stroke`（描边颜色）、`stroke-width`（描边宽度）；以及更高级的特性, 如性渐变 `linearGradient` 和径向渐变 `radialGradient` 填充图形， `transform` 属性进行平移、缩放、旋转变换， `viewBox` 属性指定 SVG 文件的坐标系统范围，使用 `<g>` 元素组合多个图形元素等。

2）CaseSVG.h 和 CaseSVG.cpp

在项目的UI和调度层，仿照先前lab的架构和教程，构建一个继承自 `Common::ICase` 的 `CaseSVG` 类，同时加入成员变量（详见代码以及注释）和渲染出 SVG 图像后 `_filterMode` 控制的简单图像处理——`SSAA`，`ApplySobel`，`ApplyBlur`，并在 `CaseSVG.cpp` 的 `OnSetupPropsUI()` 添加UI界面控制接口、完善功能函数，在 `OnRender` 渲染函数中完成响应控件的 `_recompute` 更新调用，再通过：

```
tinyxml2::XMLDocument doc;
if (doc.LoadFile(std::filesystem::path(Assets::ExampleSVGs[_SVGIdx]).string().c_str())) {
    std::cerr << "File loading error!" << GetPath(_SVGIdx) << std::endl;
}
else {
    auto const * root = doc.FirstChildElement("svg");
    renderImage(superTex, root, _width, _height, false);
}
```

即 `renderImage` 进入SVG渲染的主逻辑。

3）主逻辑：SVG 渲染

核心渲染逻辑位于 `render.cpp`，按照 ***SVG DOM → 解析属性 → 几何构造 → 曲线采样 → 光栅化 → 像素写入*** 的流程进行，每一阶段的实现以及借助函数大体如下（详见代码）：

SVG DOM

渲染器通过 `renderImage` 函数从 XML 文档中读取并遍历 `<svg>` 及其子元素。`setupContextFromRoot` 函数根据 `<svg>` 的属性如 width、height、viewBox 初始化渲染上下文，计算出缩放比例与偏移量，确保 SVG 与目标画布的正确映射；同时 `renderElementRecursive` 函数递归地遍历 SVG 元素节点，根据元素的类型如 `<rect>`、`<circle>`、`<polygon>` 等调用相应的渲染函数进行处理。

解析属性

渲染器将 SVG 中的字符串属性（坐标、颜色、路径数据等）解析成数值形式便于后续计算：一些工具函数 `isNumberChar` 用于检查字符是否是数字、`parseFloatList` 解析逗号分隔的浮点数、`parsePointList` 解析多边形的 `points` 属性、`parseHexColor` 和 `parseNamedOrRgbColor` 处理颜色数据（其中前者特殊处理十六进制的情况），`extractPaint` 提取 `fill` 和 `stroke` 颜色及其透明度。

几何构造

将基本几何形状——矩形、圆形、多边形、路径转换为更易于计算的几何表示，统一在在函数 `renderXXXElement` 中实现：`renderRectElement` 和 `renderCircleElement` 处理矩形和圆形的几何构造，`renderPolygonElement` 解析多边形的点列表并构造多边形的几何表示，`renderPathElement` 解析 SVG 中的路径数据，将 M (Move to) /L (Line to) /H (Horizontal line to)/V (Vertical line to) /Z (Closepath) /C (Cubic Bezier) /Q (Quadirc Bezzier) /A (Arc)的路径指令转换为折线段。

曲线采样

由于 SVG 中的贝塞尔曲线和椭圆弧无法直接用于光栅化，通过离散化这些曲线来生成折线。`sampleCubicBezier` 和 `sampleQuadraticBezier` 分别实现了三次和二次贝塞尔曲线的采样，`sampleSvgArc` 则处理 A 指令，采样椭圆弧。添加辅助函数 `rotate2D` 用于对坐标进行旋转，以适配曲线的几何变换。

光栅化

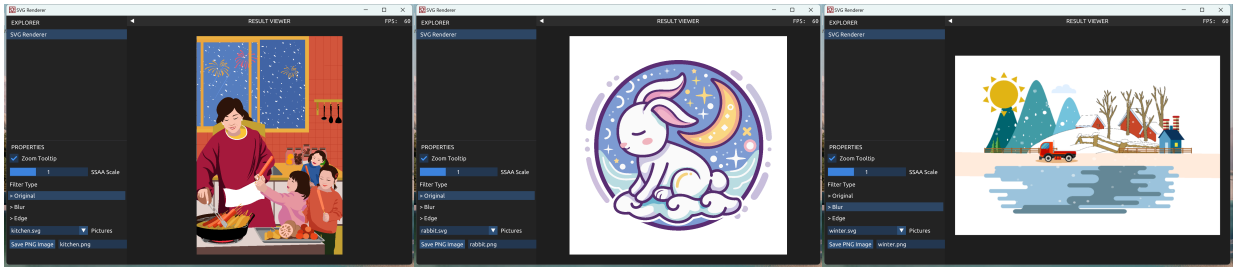
渲染器根据几何数据决定哪些像素需要被填充或描边：`buildEdgesFromPolygon` 将多边形分解为边，供扫描线算法使用；`fillPolygons` 使用扫描线算法实现多边形的填充，`strokePolyline` 根据折线的宽度构造描边，`fillRing` 用于绘制圆形及圆环。

像素写入

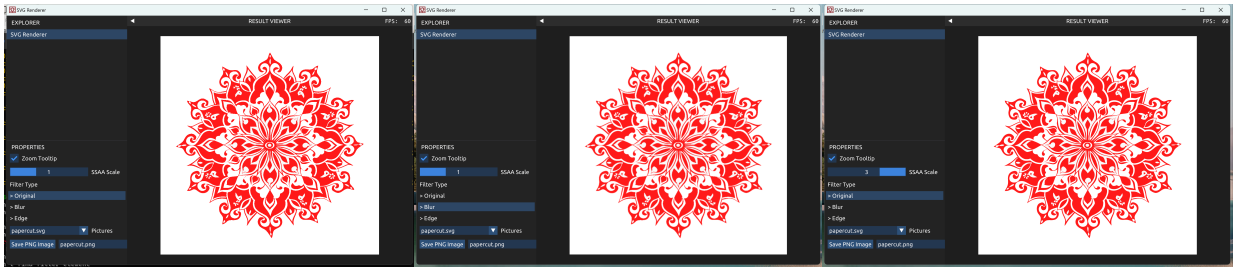
光栅化后的几何数据被转换为像素并写入画布。`svgToPixel` 将 SVG 坐标转换为像素坐标，`putPixel` 将颜色值写入对应的像素，`drawPixelInSvgSpace` 则直接在 SVG 坐标系中绘制像素点。经过上述全部流程，SVG 文件中的图形元素被逐步转换为位图、最终输出到画布上，通过`renderImage`被调用后完成图像的渲染。

3. 实现结果

能够支持基础的 SVG 格式图片渲染，如下：



和加入一些图像处理 (blur, ssaa) 后的效果：



4. 运行

解压缩后，终端进入文件夹“vcl-project”，输入指令执行：

```
xmake run svg
```